

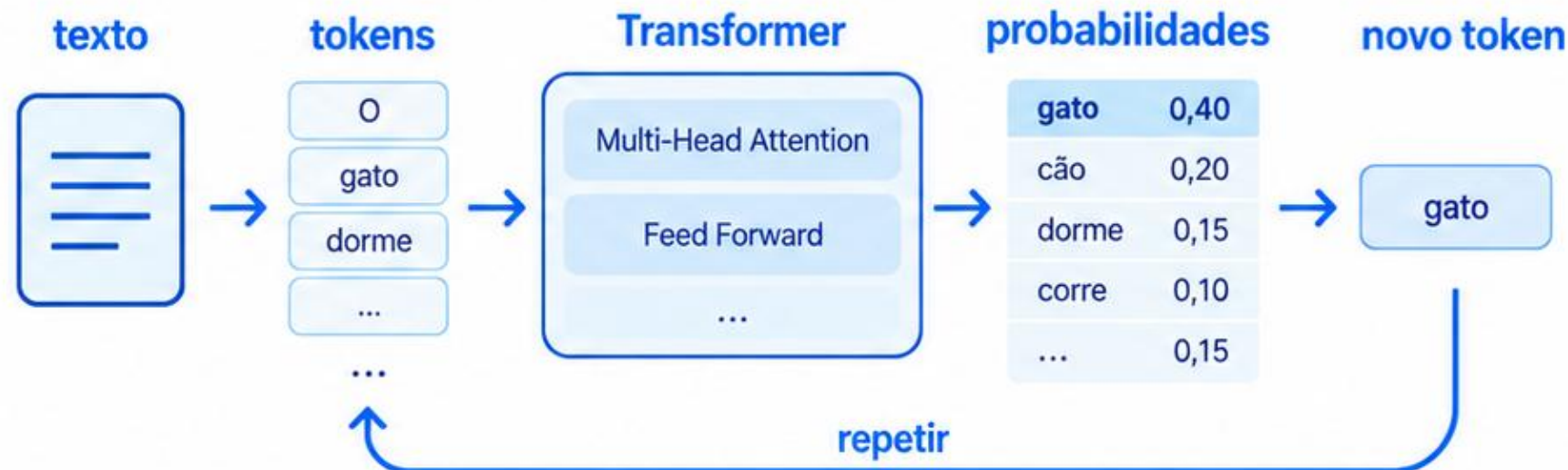
LLMs e IA Generativa

Transformers, prompting e agentes

Compreender como se gera texto e como verificar o resultado

O que vamos aprender

- 1 Relacionar tokens, embeddings e attention
- 2 Explicar o Transformer e o treino de LLMs
- 3 Construir prompts e respostas fundamentadas
- 4 Avaliar limitações e agentes com ferramentas



Compreender

Interpretar o mecanismo



Aplicar

Dar contexto e critérios



Verificar

Testar respostas e ações

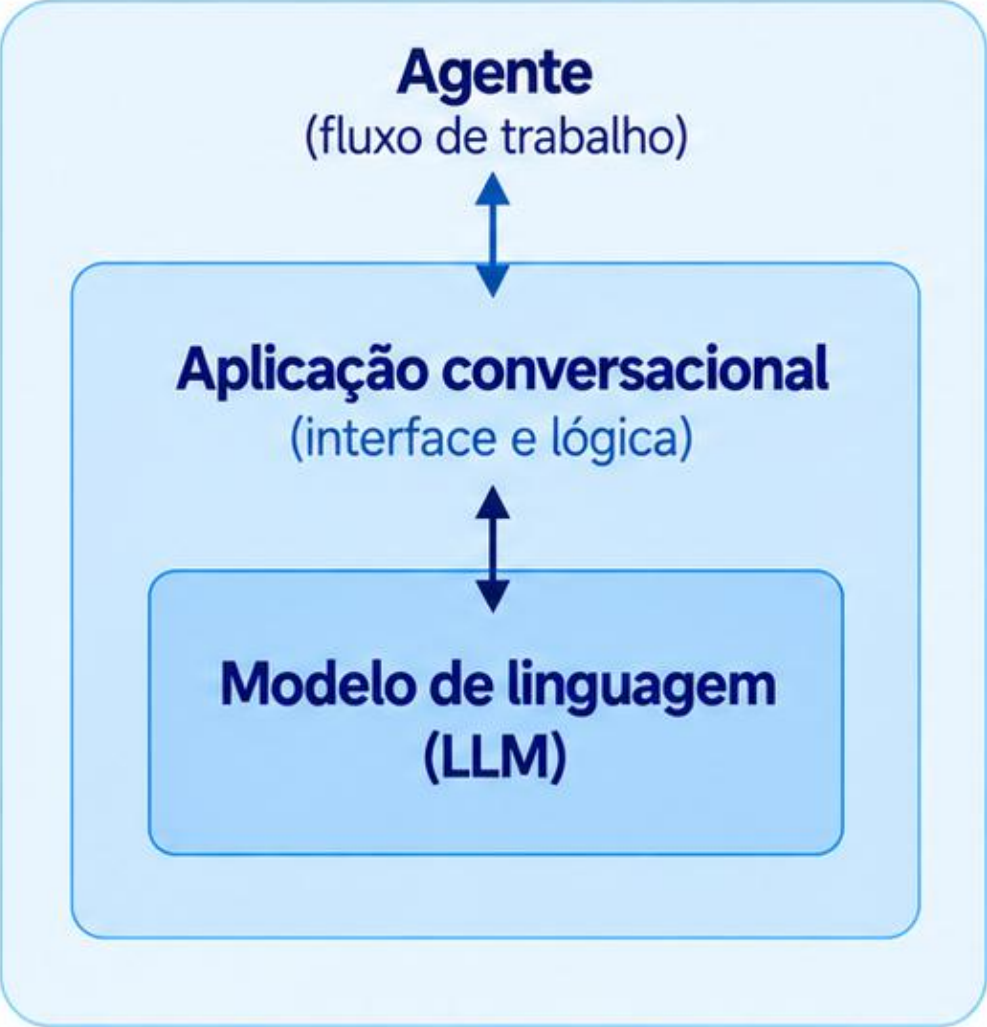
IA generativa e modelos de linguagem

IA generativa produz novos conteúdos condicionados por entradas e padrões aprendidos.

Modalidades de entrada e saída

Modalidade	Entrada (exemplo)	Saída
Texto	Instrução	Resumo
Código	Especificação	Função
Imagem	Descrição	Ilustração
Áudio	Texto	Fala

Componentes e relação



Modelos de linguagem

LLM: modelo de linguagem de grande escala.

Um modelo, uma aplicação conversacional e um agente são componentes distintos.

Geração de texto

A geração de texto estima continuações plausíveis.

Plausibilidade não garante verdade.

Evolução até aos LLMs

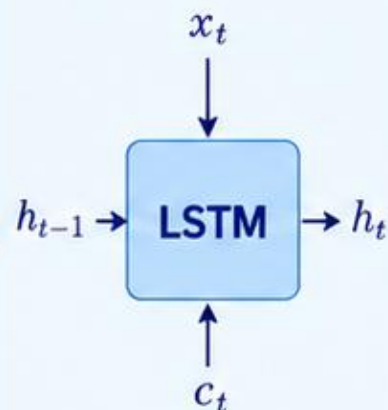
Modelos n-grama

contexto local
por contagens

$$P(w_i | w_{i-2}, w_{i-1}) \approx \frac{C(w_{i-2}, w_{i-1}, w_i)}{C(w_{i-2}, w_{i-1})}$$

RNN e LSTM

estado recorrente
para sequências



Attention

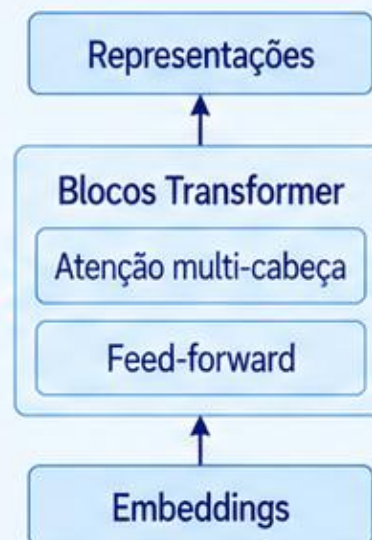
acesso ponderado
a representações

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$Q \rightarrow$ Atenção
 $K \rightarrow$
 $V \rightarrow$

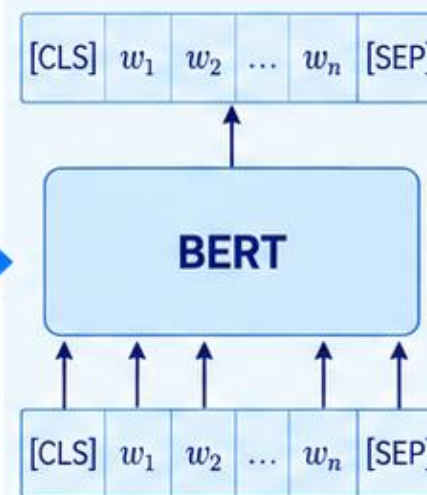
Transformer, 2017

arquitetura baseada
em atenção



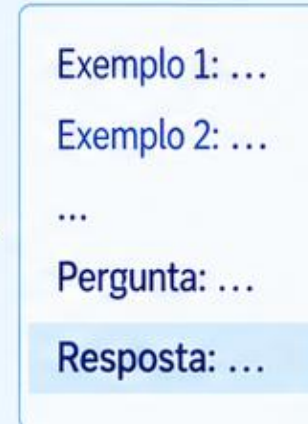
BERT, 2018

representações
bidirecionais



GPT-3, 2020

aprendizagem
com exemplos
no contexto



InstructGPT, 2022

seguimento de
instruções com
feedback humano



Marcos selecionados, não uma cronologia exaustiva.

**Escala de dados, computação e treino contribuiu para novas capacidades.
A arquitetura, por si só, não explica todos os resultados.**

Tokens e tokenização

O que são tokens?

- Um token pode representar uma palavra, subpalavra, sinal ou fragmento.
- IDs de tokens são índices, não significados.
- O número de tokens difere do número de palavras.

Exemplo de tokenização

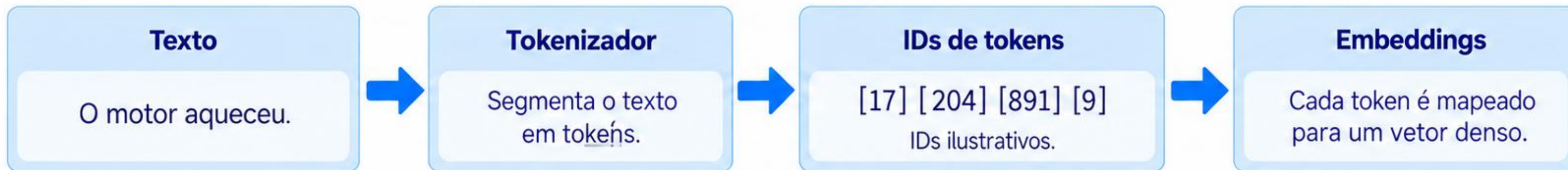


Exemplo de divisões possíveis

Texto	Possível divisão
engenharia	engen + haria
2026	20 + 26

Divisões ilustrativas, dependentes do tokenizador.

Pipeline de processamento



A janela de contexto limita a quantidade de tokens processados.

O limite exato depende do modelo e da configuração.

Embeddings e informação de posição

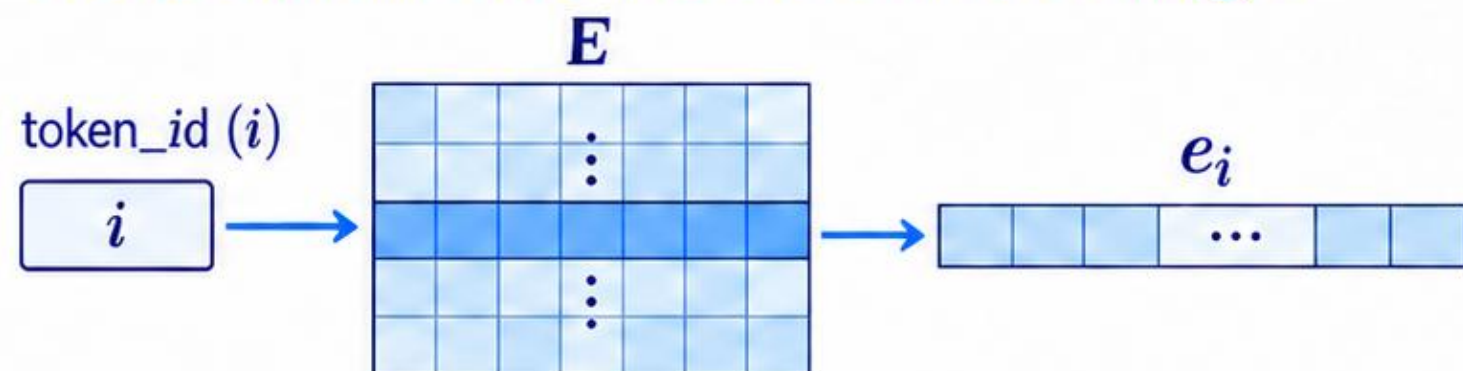
Embeddings de tokens

$$E \in \mathbb{R}^{|V| \times d}$$

onde $|V|$ é o tamanho do vocabulário e d é a dimensão.

$$e_i = E[\text{token}_i].$$

Consulta na matriz de embeddings



- Tokens são representados por vetores aprendidos.
- As camadas produzem representações dependentes do contexto.

Exemplos de dependência do contexto



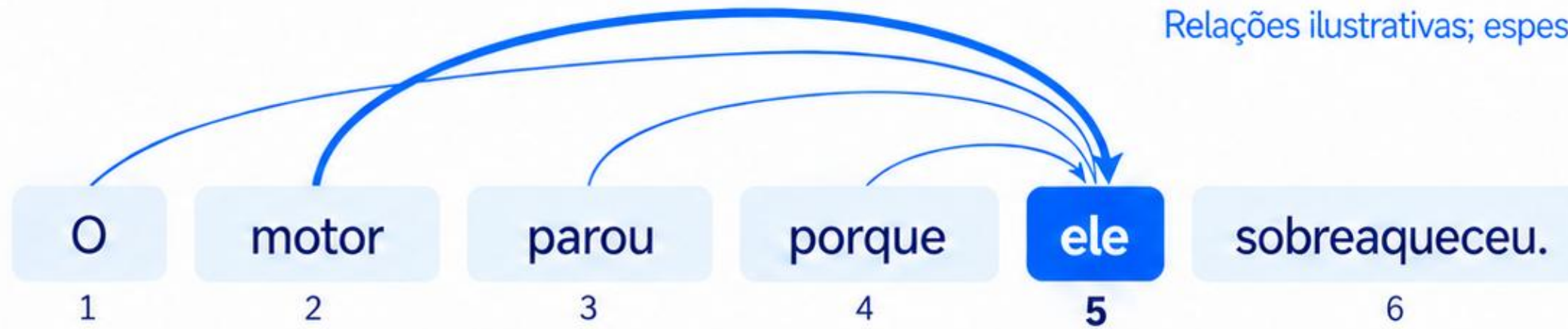
O embedding inicial do mesmo token é igual; as representações contextuais podem diferir.

Informação de posição

- A ordem deve ser codificada.
- No Transformer original: representação de entrada = embedding + codificação posicional.
- Existem alternativas, como embeddings posicionais aprendidos e posições rotativas.

Distâncias entre vetores podem refletir relações aprendidas, mas não constituem uma base de factos verificados.

Self-attention: relacionar tokens



Query: o que a posição procura

Representa a pergunta que a posição faz aos outros tokens para encontrar informação relevante.

Key: como cada posição pode ser comparada

Representa a forma como cada posição é descrita, para ser comparada com a query de outra posição.

Value: informação a combinar

Representa a informação que será combinada, ponderada pelos pesos de atenção.

Projeções lineares

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

Cada posição calcula uma combinação ponderada dos valores

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Self-attention usa a mesma sequência para formar Q , K e V

A mesma sequência de tokens é projetada para obter as matrizes Q , K e V .

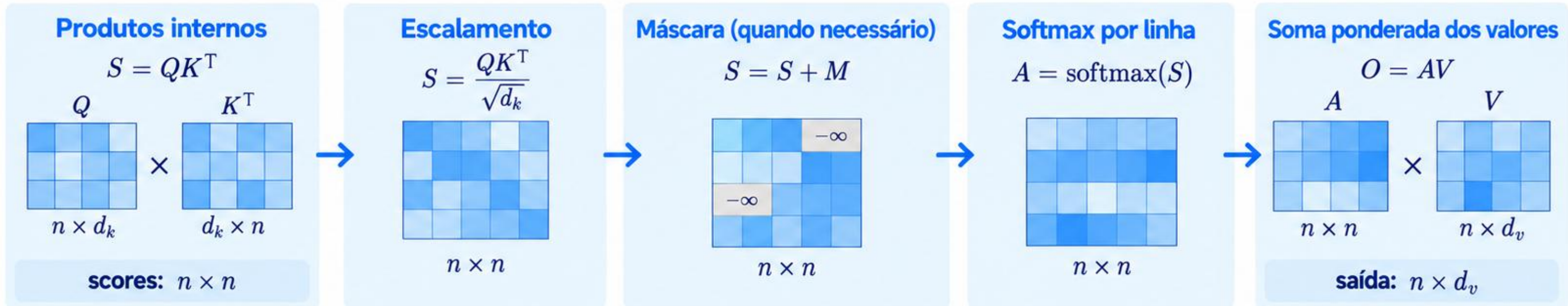
Um peso de atenção não é uma explicação causal suficiente da resposta do modelo.

Scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

- A divisão por $\sqrt{d_k}$ controla a escala dos produtos internos.
- Cada linha da softmax soma 1.
- Máscaras bloqueiam posições não permitidas.

Passos do cálculo



Dimensões

- Q, K : $n \times d_k$
- V : $n \times d_v$
- scores (QK^T): $n \times n$
- saída ($\text{Attention}(Q, K, V)$): $n \times d_v$

Nota final

Na atenção densa, a matriz de relações entre n tokens tem n^2 entradas. Implementações eficientes podem reduzir a memória materializada.

Attention: exemplo numérico

1. Dados do exemplo

$q = (1, 0)$
 $k_1 = (1, 0)$
 $k_2 = (0, 1)$
 $d_k = 2$
 $v_1 = (2, 0)$
 $v_2 = (0, 4)$

(query)
(chave 1)
(chave 2)
(dimensão das chaves)
(valor 1)
(valor 2)

2. Produto interno e escala

Calculamos os produtos internos entre a query e cada chave e aplicamos a escala $1/\sqrt{d_k}$.

$$q \cdot k_1 = 1 \quad q \cdot k_2 = 0$$
$$\frac{1}{\sqrt{d_k}} = \frac{1}{\sqrt{2}} \approx 0,7071$$

scores escalados:
(0,7071; 0)

3. Pesos com softmax

Aplicamos a função softmax aos scores escalados:

$$\alpha_i = \frac{\exp(score_i)}{\sum_j \exp(score_j)}$$

Pesos (aproximados):
(0,6698; 0,3302)

Os pesos somam 1:
 $0,6698 + 0,3302 = 1$

4. Tabela de resultados

Chave (k_i)	Score escalado	Peso (α_i)
$k_1 = (1, 0)$	0,7071	0,6698
$k_2 = (0, 1)$	0	0,3302

5. Combinação ponderada dos valores

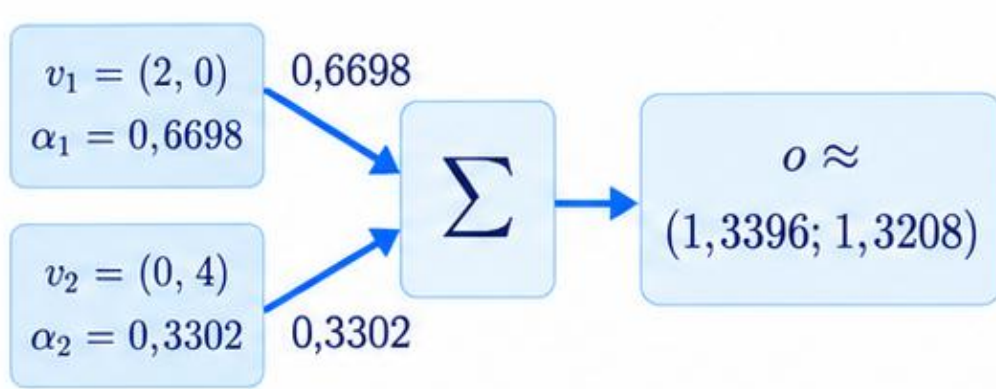
A saída da attention é a soma ponderada dos valores:

$$o = \sum_i \alpha_i v_i$$

Substituindo os valores:

$$o = 0,6698(2, 0) + 0,3302(0, 4) \approx (1,3396; 1,3208)$$

6. Diagrama da combinação ponderada



A saída combina os valores V , não os vetores K .

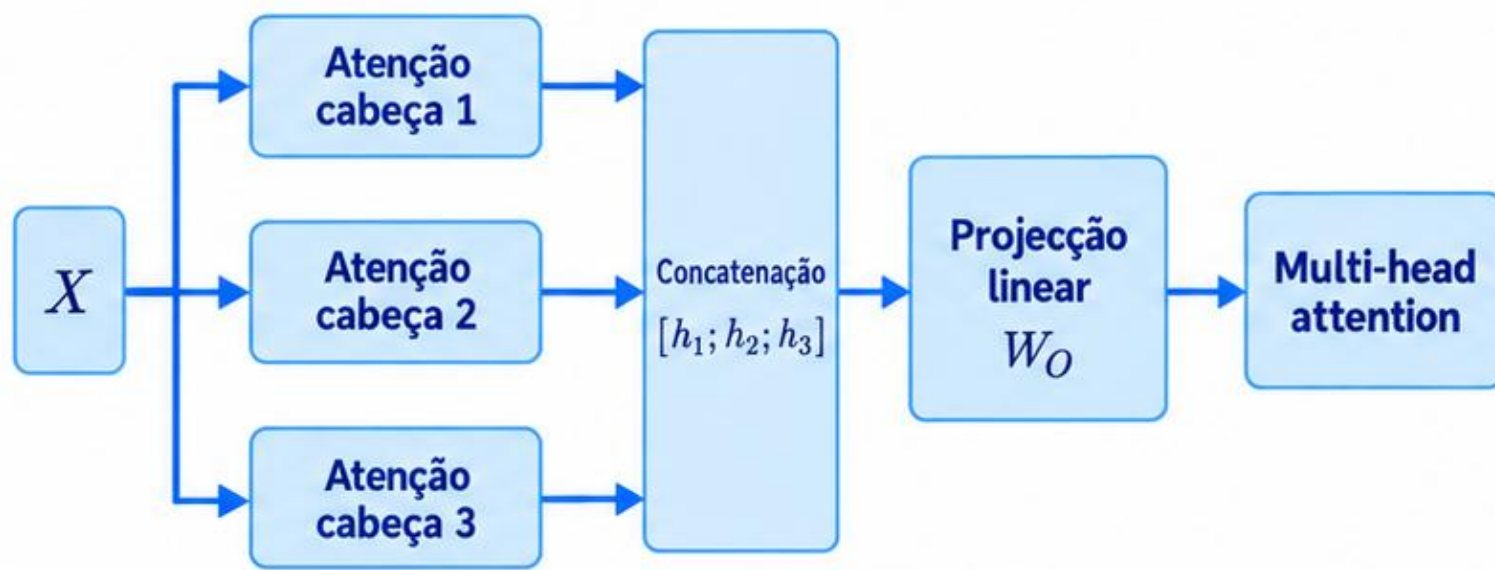
Os pesos somam 1.

Exemplo didático com valores arredondados.

Multi-head attention e bloco Transformer

Multi-head attention

Calcula várias atenções em paralelo, concatena os resultados e aplica uma projecção linear.



Três cabeças apenas para ilustrar.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O$$

Cabeças aprendem diferentes projecções

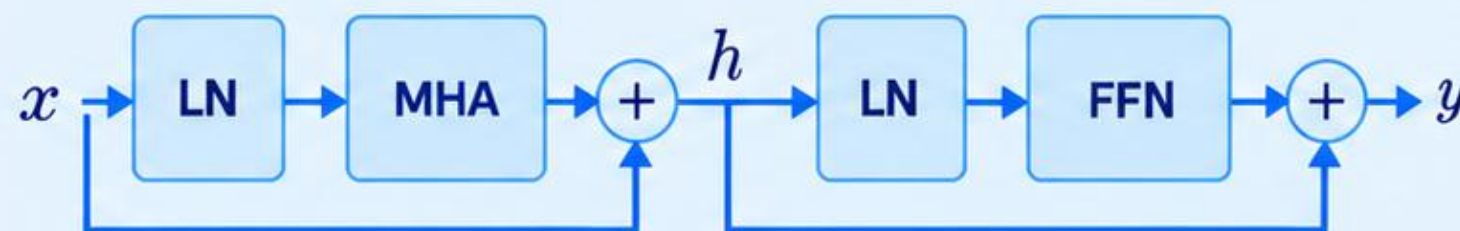
Cada cabeça pode focar-se em diferentes padrões de dependências, o que permite captar relações enriquecidas entre as posições.

Bloco Transformer

Combina multi-head attention e uma rede feedforward, cada um com ligações residuais e normalização (pré-normalização).

Esquema com pré-normalização

A posição das normalizações varia entre arquiteturas.



$$h = x + \text{MHA}(\text{LN}(x))$$

$$y = h + \text{FFN}(\text{LN}(h))$$

Feedforward transforma cada posição

Uma rede totalmente ligada, aplicada de forma independente a cada posição, aumenta a capacidade de modelação.

Ligações residuais e normalização apoiam o treino

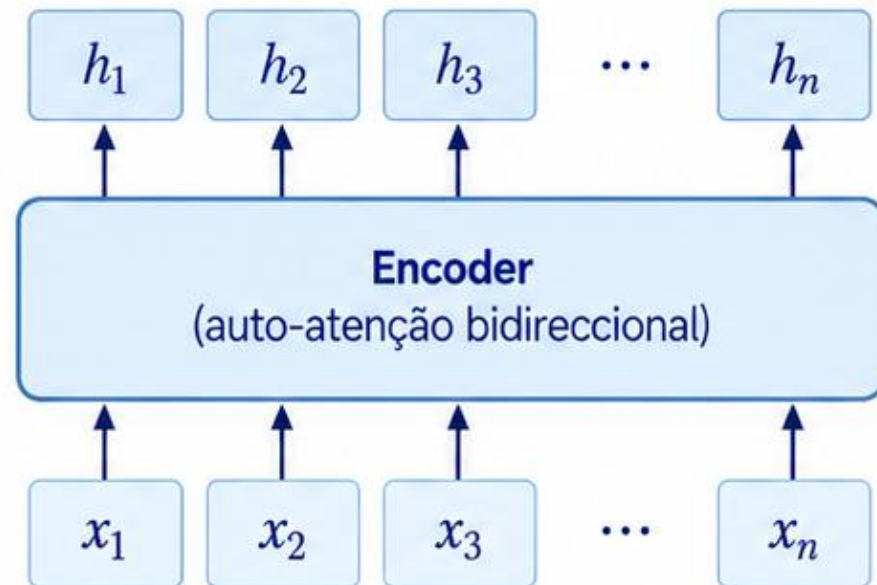
As ligações residuais facilitam o fluxo de gradientes e a normalização estabiliza o treino de modelos profundos.

Encoder, decoder e máscara causal

Encoder: representações bidireccionais

Ex.: BERT

O encoder processa toda a sequência com atenção bidireccional, produzindo representações contextuais para cada token.



Representações contextuais:

$$h = [h_1, h_2, \dots, h_n]$$

Decoder: previsão do próximo token

Ex.: GPT

O decoder usa uma máscara causal para impedir a atenção a tokens futuros, prevendo o próximo token de forma autorregressiva.

Máscara causal (3×3)

Posição que consulta	Posição consultada		
	1	2	3
1	1	0	0
2	1	1	0
3	1	1	1

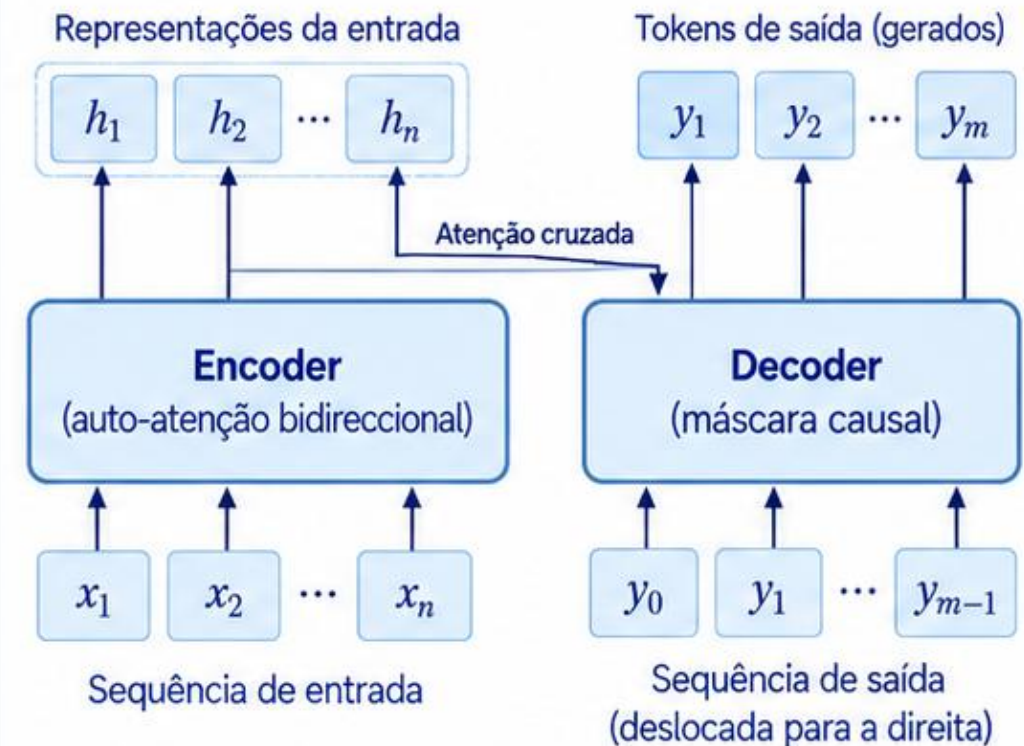
1: atenção permitida
0: atenção bloqueada

Na previsão autorregressiva, uma posição não acede aos tokens futuros.

Encoder-decoder: entrada e saída

Ex.: tradução

O encoder-decoder combina um encoder para representar a entrada com um decoder que gera a saída, acrescentando atenção à representação da entrada.



A escolha depende da tarefa. Nem todos os Transformers são modelos generativos autorregressivos.

Pré-treino: prever o próximo token

Modelação da linguagem

O modelo define uma distribuição conjunta para a sequência de tokens como o produto de distribuições condicionais:

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_{<t})$$

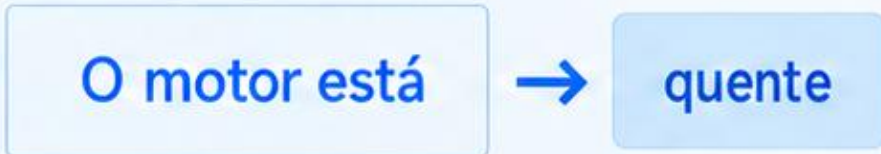
Objetivo de treino

Maximizamos a log-verosimilhança, equivalentemente minimizamos a perda de entropia cruzada:

$$\mathcal{L} = - \sum_{t=1}^T \ln P_{\theta}(x_t \mid x_{<t})$$

Exemplo ilustrativo

Dada a sequência “O motor está”, o modelo aprende a prever o próximo token “quente”.



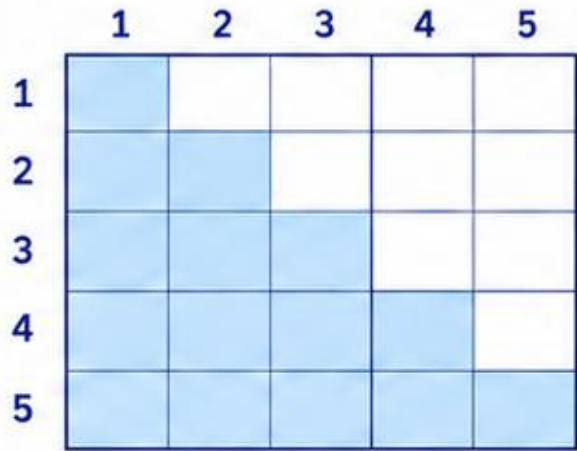
Alvos de treino a partir de texto deslocado

Em cada posição, o alvo é o token seguinte da sequência.

Entradas	O	motor	está
Alvos	motor	está	quente

Máscara causal

Durante o treino, posições podem ser calculadas em paralelo com máscara causal.



Geração sequencial

Durante a geração, os tokens são produzidos sequencialmente.



Aprendizagem e optimização

- **Aprendizagem auto-supervisionada:** o texto fornece os alvos.
- *Backpropagation* calcula gradientes; o otimizador ajusta os parâmetros.

A distribuição dos dados e o objetivo de treino condicionam o que o modelo aprende.

Pós-treino e seguimento de instruções



Nem todos os modelos seguem a mesma receita.

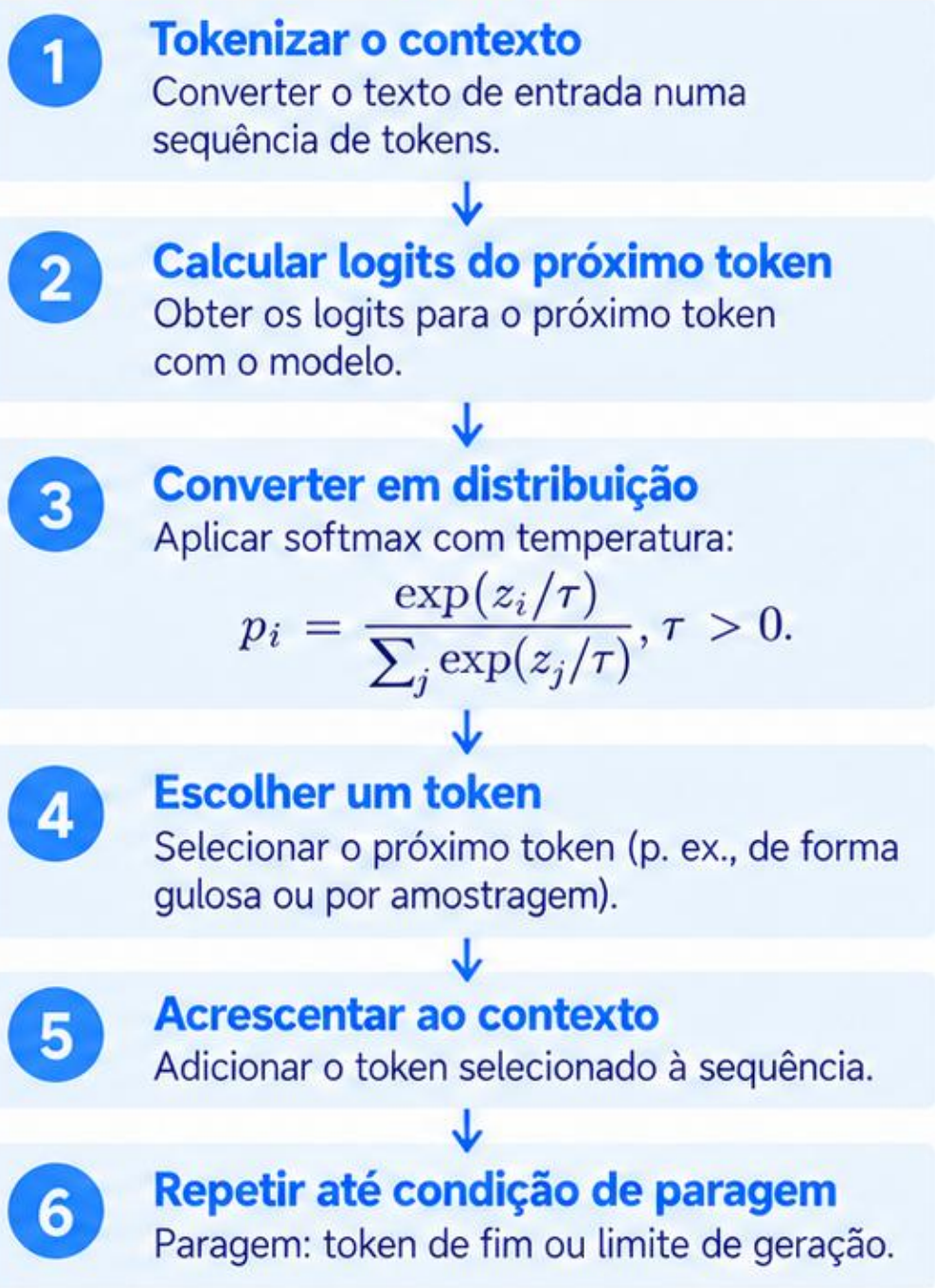
Treino
altera pesos.

Prompting
altera o contexto.

Recuperação
acrescenta informação externa.

Geração e estratégias de decodificação

Algoritmo de geração



Distribuição de probabilidade

Softmax com temperatura:

$$p_i = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}, \tau > 0.$$

Efeito da temperatura

Exemplo: logits $z = (2; 1; 0)$

τ	P(A)	P(B)	P(C)
0,5	0,867	0,117	0,016
1,0	0,665	0,245	0,090
2,0	0,506	0,307	0,186

Probabilidades arredondadas.

Estratégias de decodificação

Estratégia	Descrição
gulosa	argmax
temperatura	ajusta concentração
top-p	restringe amostragem a um núcleo de probabilidade

Considerações importantes

- Temperatura mais baixa tende a concentrar a distribuição.
- Maior diversidade não equivale a maior correção.

Prompting: estrutura de uma tarefa

Objetivo

Descrever claramente o que se pretende que o modelo faça.

Contexto e dados

Fornecer o contexto necessário e os dados relevantes.

Restrições

Indicar o que o modelo não deve fazer e quais são os limites da resposta.

Formato de saída

Especificar a estrutura e o formato da resposta pretendida.

CrITÉRIOS de verificação

Definir como avaliar se a resposta cumpre o objetivo.

Exemplo de prompt completo

“Analisa o relatório fornecido.

Identifica apenas falhas explicitamente descritas.

Devolve uma tabela com falha, evidência textual e ação sugerida.

Se a informação não constar do relatório, escreve não indicado.

Não inventes medições.”

Delimitar documentos de entrada

- Indicar quais os documentos a utilizar.
- Especificar o formato e, se relevante, as secções a considerar.
- Evitar ambiguidade sobre a fonte de informação.

Especificar o público e o nível de detalhe

- Indicar quem é o destinatário da resposta.
- Definir o nível de detalhe adequado.
- Ajustar a linguagem ao público.

Dar exemplos quando o formato for ambíguo

- Fornecer um exemplo do formato esperado.
- Incluir exemplos de casos típicos, se útil.
- Tornar explícita a estrutura da resposta.

Testar com casos representativos

- Experimentar o prompt com diferentes casos.
- Verificar se a resposta cumpre o objetivo.
- Ajustar o prompt quando necessário.

Um prompt claro melhora a especificação, mas não garante obediência nem exatidão.

Zero-shot, few-shot e decomposição

Tipos de abordagem

Abordagem	Descrição	Exemplo de uso
Zero-shot	instrução sem exemplos	classificar sentimento
Few-shot	exemplos no contexto	mostrar formato e rótulos
Decomposição	subtarefas verificáveis	extrair, calcular, redigir

Exemplo de classificação

- Exemplo:**
O motor funciona normalmente → **normal**
- Exemplo:**
O motor vibra excessivamente → **anomalia**
- Novo:**
Há ruído irregular no rolamento → **?**

Notas importantes

- Exemplos devem cobrir casos relevantes e não contradizer a instrução.
- Pedir resultados intermédios verificáveis, como cálculos ou evidências.
- Few-shot no contexto não atualiza os pesos.

Avaliar se a decomposição melhora a tarefa concreta.

RAG: geração com recuperação



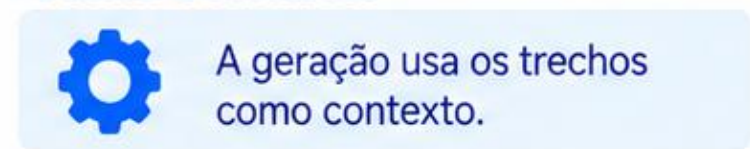
Preparação do índice



Recuperação procura informação externa



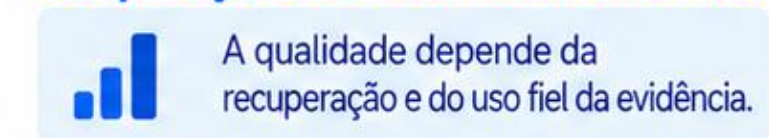
A geração usa os trechos como contexto



As fontes devem ser rastreáveis



A qualidade depende da recuperação e do uso fiel da evidência



Exemplo de engenharia



Pseudocódigo (fluxo simplificado)

```
docs = carregar_documentos()
chunks = segmentação(docs)
embs = embeddings(chunks)
índice = construir_índice(embs)

trechos = pesquisar(pergunta, índice)
contexto = montar_contexto(trechos)
resposta = LLM(pergunta, contexto)
devolver resposta
```

RAG pode reduzir respostas sem suporte, mas não elimina alucinações.

Hallucinations e outras limitações

Definição

Alucinação: conteúdo apresentado como factual sem suporte adequado ou em contradição com a evidência.

Causas

- Objetivo de geração não é verificação factual.
- Contexto insuficiente ou ambíguo.
- Falhas de recuperação ou raciocínio.
- Conhecimento aprendido pode estar desatualizado.

Exemplos

- 1 Citação inventada**
“Segundo Silva (2021)...” (referência inexistente).
- 2 Resultado numérico incorreto**
“A raiz quadrada de 2 é 3.” (valor errado).
- 3 Leitura incorreta de manual**
Interpretação errada de uma instrução do manual.

Mitigações

- Verificar fontes originais.
- Usar ferramentas para cálculos.
- Permitir abstenção.
- Comparar respostas com evidência.

Fluência, detalhe e confiança verbal não demonstram correção.

Avaliação de aplicações com LLMs

Tarefas, critérios e testes

Tarefa	Critério	Teste
extração	fidelidade	comparar campos com fonte
resumo	cobertura e ausência de invenções	grelha de avaliação
código	comportamento correto	testes automatizados
RAG	evidência suficiente	verificar trechos citados
agente	conclusão e ações válidas	registros de execução

Taxa de sucesso

$$taxa\ de\ sucesso = \frac{casos\ aprovados}{casos\ avaliados}$$

Metodologia de avaliação

- Usar conjunto fixo de casos normais, difíceis e adversariais
- Comparar versões de prompt/modelo
- Medir latência e custo
- Revisão humana onde os critérios automáticos são insuficientes

Agentes com LLMs e ferramentas

Instruções e limites

- Objetivo da tarefa.
- Regras e restrições.
- Formato das respostas.
- Limites de segurança e permissões.

Ferramentas e permissões

- Conjunto de ferramentas disponíveis.
- Descrição das funcionalidades.
- Permissões e limites de utilização.
- Validação das chamadas.

Estado da tarefa

- Histórico de mensagens.
- Resultados de ferramentas.
- Informação relevante do contexto.
- Planeamento intermédio.

CrITÉrios de paragem

- Objetivo alcançado.
- Resposta final produzida.
- Limite de passos ou tempo.
- Condições de erro.

Objetivo



Tarefa do utilizador

LLM decide



Escolhe a próxima ação

Ferramenta



Ferramenta externa

Observação



Resultado da ferramenta

LLM loop



Atualiza o estado e decide novamente

Resultado



Resposta final ao utilizador

O LLM propõe chamadas estruturadas

```
{
  "ferramenta": "calculadora",
  "argumentos": {
    "expressao": "2 * (3 + 4)"
  }
}
```

A aplicação valida e executa

```
if ferramenta in permitidas:
    validar(argumentos)
    resultado = executar(ferramenta, argumentos)
else:
    erro("ferramenta não permitida")
```

O resultado da ferramenta regressa ao contexto

```
{
  "ferramenta": "calculadora",
  "resultado": "14"
}
```

Exemplos de ferramentas



calculadora



consulta documental



execução de testes

Um agente combina o modelo com um ciclo de controlo. Ter ferramentas não garante planeamento correto.

Ciclo de um agente: exemplo

Diagnosticar uma falha descrita num relatório e preparar uma recomendação.

1. Objetivo e tarefa

Objetivo: diagnosticar uma falha descrita num relatório e preparar uma recomendação.

Tarefa: transformar o relato do problema num plano de ação fundamentado.

2. Ciclo de um agente



3. Pseudocódigo

```
até concluir ou atingir limite:  
  obter próxima ação do modelo  
  validar ferramenta e argumentos  
  executar ação permitida  
  registar resultado  
  atualizar contexto  
se faltar evidência:  
  pedir informação ou declarar limite
```

4. Boas práticas

- Separar observações de hipóteses.
- Definir limite de passos e custo.
- Confirmar ações com impacto externo conforme as permissões.
- A conclusão deve permitir verificar o percurso seguido.

Riscos e controlo da aplicação

risco	exemplo	controlo
Prompt injection	documento tenta substituir instruções	tratar conteúdo externo como dados
privacidade	dados sensíveis no contexto	minimização e controlo de acesso
ação incorreta	ferramenta altera registo errado	validação e permissões restritas
Erro propagado	saída errada alimenta tarefa seguinte	verificações entre etapas

- **Não confiar no modelo como único controlo de segurança**
- **Registrar chamadas e resultados**
- **Isolar execução de código**
- **Definir intervenção humana para ações críticas**

Os controlos pertencem à arquitetura da aplicação, além do prompt.

Exercício: um assistente técnico verificável

Contexto

Manual didático: “substituir filtro a cada 500 horas.
Não fornece valores de temperatura.”

Resposta esperada

Intervalo: 500 horas, com referência ao trecho.

Escreva um prompt para extrair o intervalo e citar a evidência

Extraí o intervalo de substituição do filtro do manual.
Devolve o valor numérico, a unidade e cita literalmente
o trecho de evidência.

Como responder à pergunta temperatura máxima?

“Qual é a temperatura máxima de operação?”

Temperatura: não indicada na fonte.

Quando usar RAG, calculadora ou agente?

- **RAG** para encontrar documentação.
- **Calculadora** para contas.
- **Agente** para coordenar ações.

Defina dois testes de avaliação

1. **Testar evidência correta** (verifica se o assistente cita o trecho exato do manual).
2. **Testar abstenção perante informação ausente** (verifica se o assistente não inventa valores).

Transformer combina representações. **LLM** gera tokens.
A **aplicação** acrescenta contexto, ferramentas e verificação.